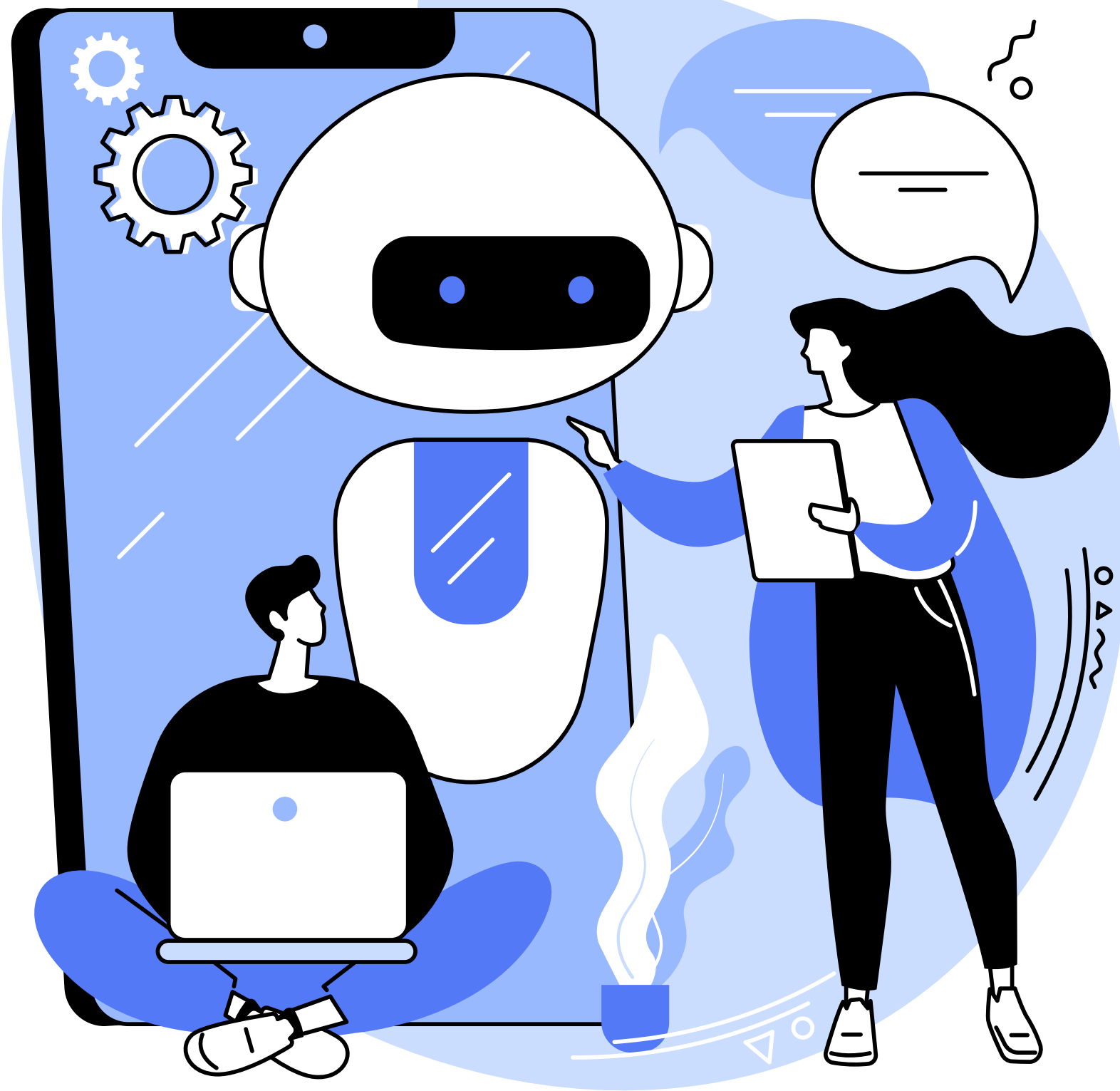
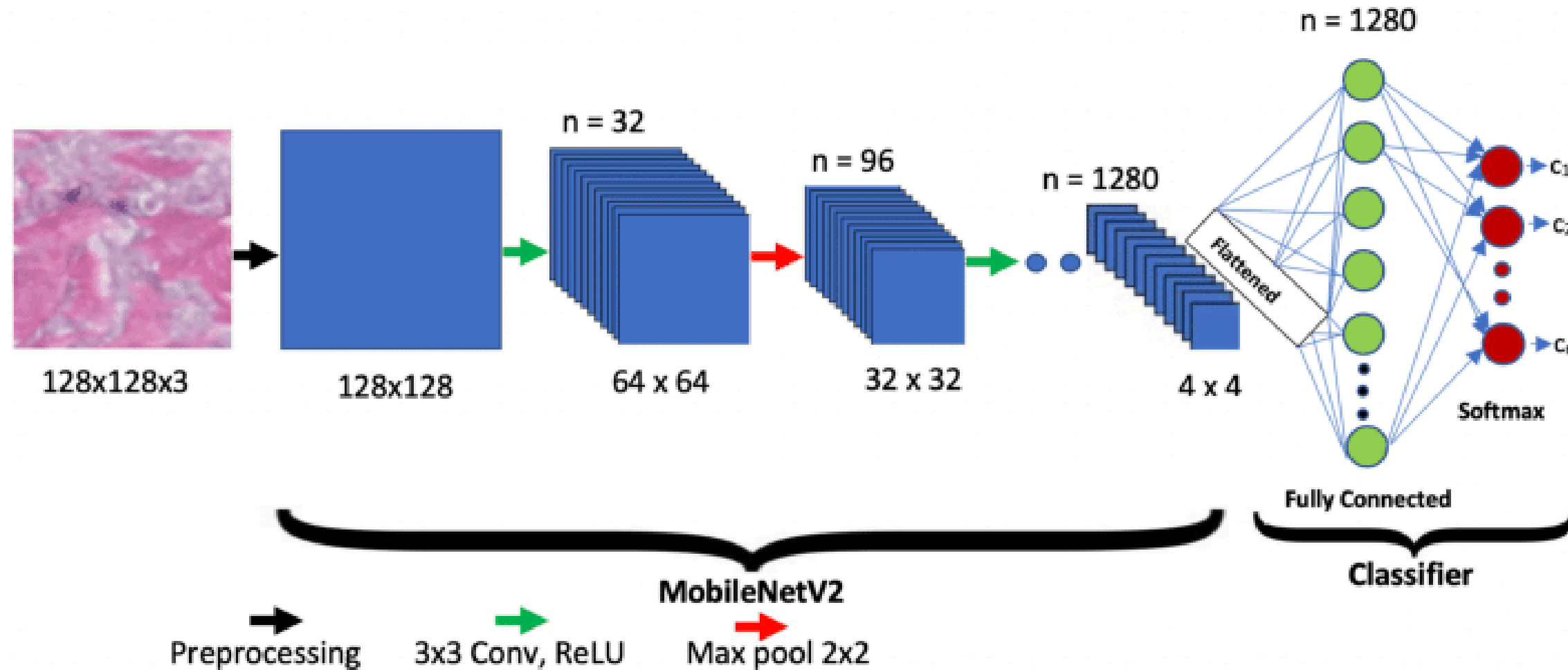




MobileNet on C++

- Geçen sunumda PyTorch ile MobileNetV2 modelini kurup ONNX yapısını analiz ettim ve C++ tarafında modelin katman yapılarını oluşturup ONNX modelinden ağırlıkları çektim.

- Bu sunumda yapıyı gerçek bir input alıp tüm katmanlardan geçirerek çıktı üreten bir forward computation sistemine dönüştürmeyi hedefledim.



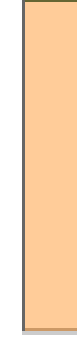
TENSOR

- MobilenetV2 için üç boyutlu tensör yapısı tasarladım: kanal sayısı, yükseklik ve genişlik değerlerini tutuyor.
- Tüm katmanların giriş ve çıkışları bu Tensor üzerinden taşıyor.

```
1  #ifndef TENSOR_H
2  #define TENSOR_H
3
4  #include <cstring>
5  #include <algorithm>
6
7  struct Tensor {
8      int c;
9      int h;
10     int w;
11     float* data; // size = c * h * w
12
13     // Default ctor
14     Tensor() : c(0), h(0), w(0), data(nullptr) {}
15
16     // Normal ctor
17     Tensor(int c_, int h_, int w_) : c(c_), h(h_), w(w_) {
18         data = new float[c * h * w];
19     }
20
21     // Deep copy constructor
22     Tensor(const Tensor& other) : c(other.c), h(other.h), w(other.w) {
23         if (other.data) {
24             data = new float[c * h * w];
25             std::memcpy(data, other.data, sizeof(float) * c * h * w);
26         } else {
27             data = nullptr;
28         }
29     }
30
31     // Copy assignment operator
32     Tensor& operator=(const Tensor& other) {
33         if (this == &other) return *this;
34
35         if (data) delete[] data;
36
37         c = other.c;
38         h = other.h;
39         w = other.w;
40
41         if (other.data) {
42             data = new float[c * h * w];
43             std::memcpy(data, other.data, sizeof(float) * c * h * w);
44         } else {
45             data = nullptr;
46         }
47
48         return *this;
49     }
50 }
```

- Her Conv2d katmanı input olarak bir Tensor alır, output olarak yeni bir Tensor üretir
- ReLU6, Residual Add, GAP, Linear gibi tüm katmanlar Tensor üzerinde işlem yapar
- Modelin başında tensor resim verisini tutar, ilerledikçe bu tensor özellik haritasına dönüşür
- Sonda Linear katmandan çıkan tensor 1000 sınıf skorudur.

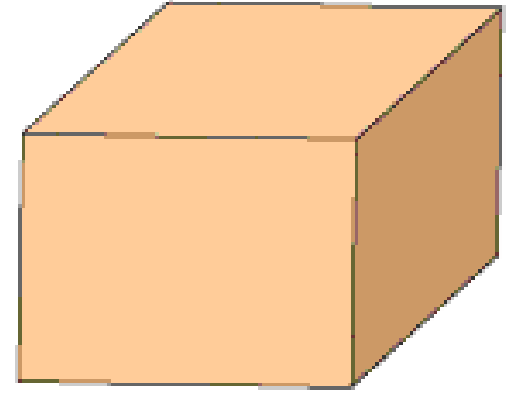
Dimensions of Tensor



1 d - Tensor



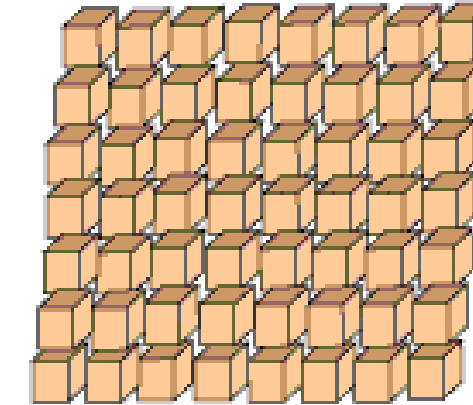
2 d - Tensor



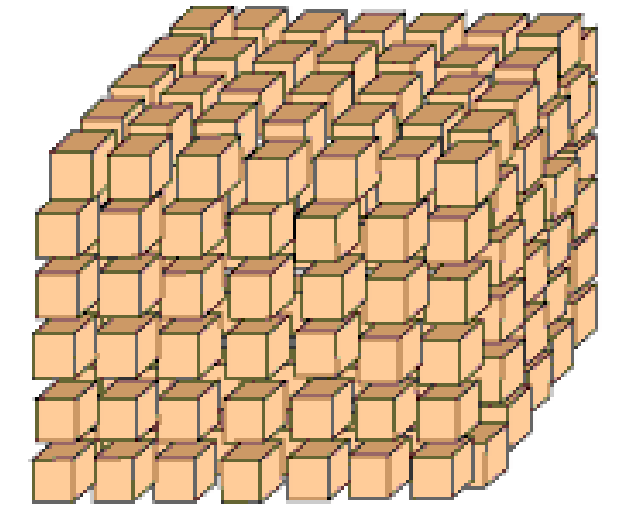
3 d - Tensor



4 d - Tensor



5 d - Tensor

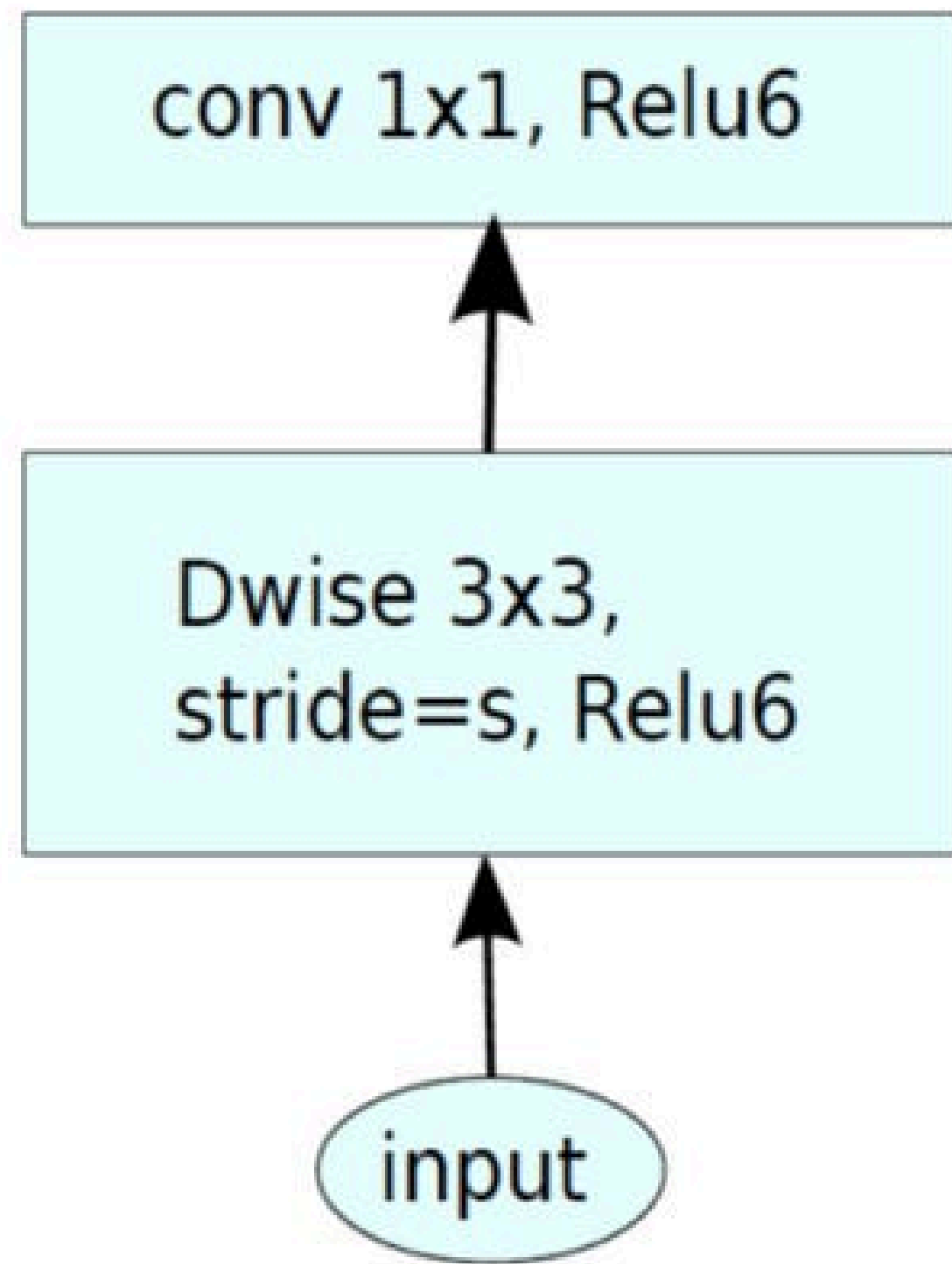


6 d - Tensor

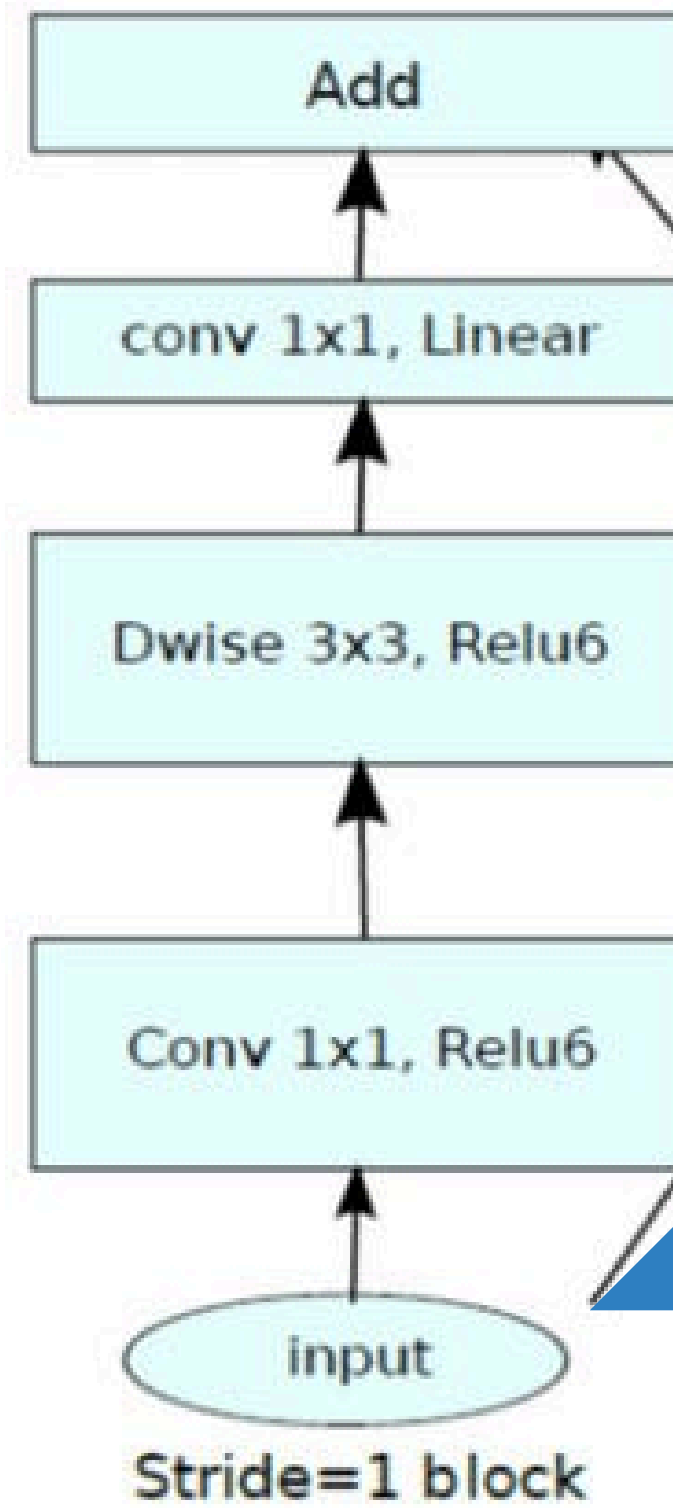
Residual Add

- Bu sınıf, bir bloğun girişinden gelen özellik haritasını tutarak, bloğun çıkışı ile element wise toplama yapan bir residual skip connection oluşturuyor.
- Forward sırasında model, normal akıştan gelen çıktıya residual olarak işaretlenen önceki katmanın çıktısını ekleyip MobileNetV2 inverted residual yapısını tamamlıyor.

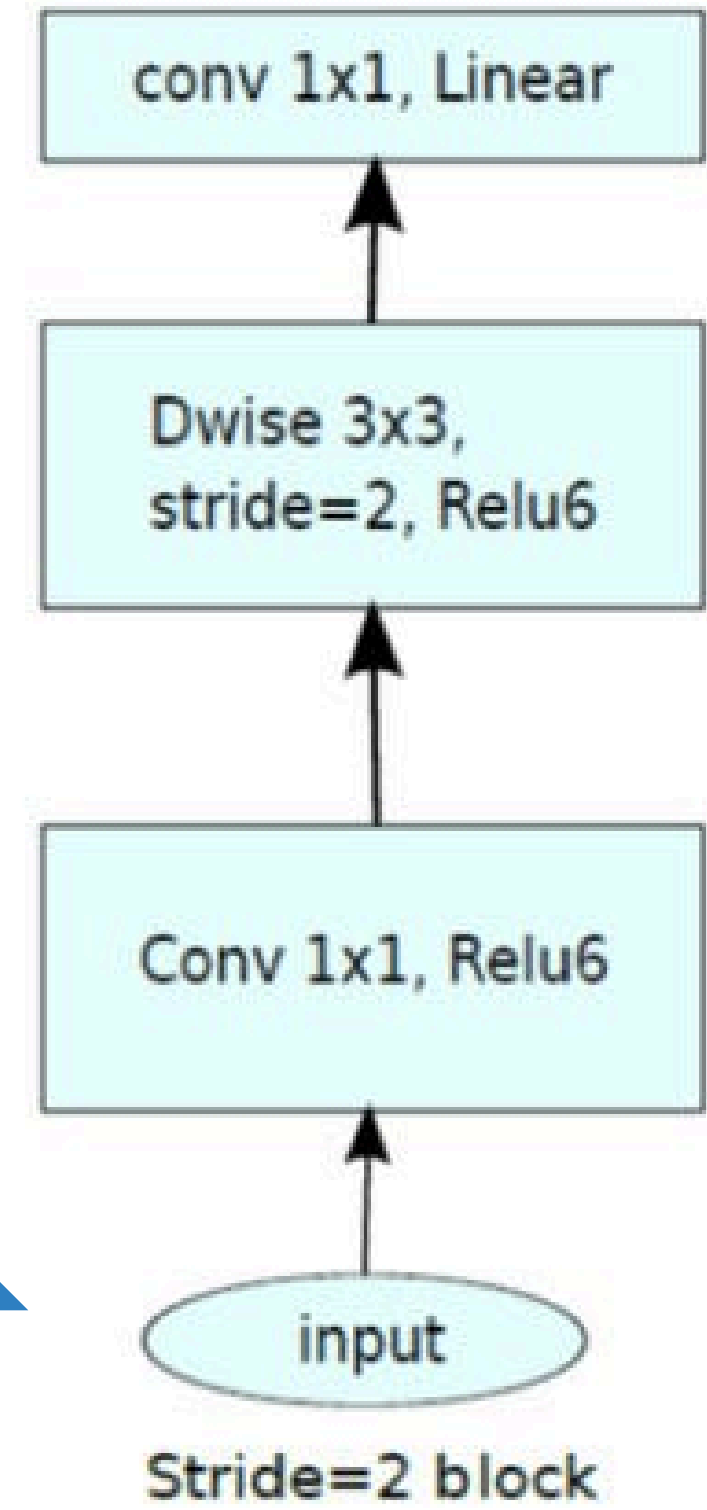
```
mobileNetCuda > C++ ResidualAdd.cpp
1  #include "ResidualAdd.h"
2  #include "Tensor.h"
3  #include <cassert>
4
5  Tensor residual_add_forward(const Tensor& x, const Tensor& skip) {
6      assert(x.c == skip.c);
7      assert(x.h == skip.h);
8      assert(x.w == skip.w);
9
10     Tensor y(x.c, x.h, x.w);
11     int size = x.c * x.h * x.w;
12
13     for (int i = 0; i < size; i++) {
14         y.data[i] = x.data[i] + skip.data[i];
15     }
16
17     return y;
18 }
19
```

MobileNetV1



Stride=1 block



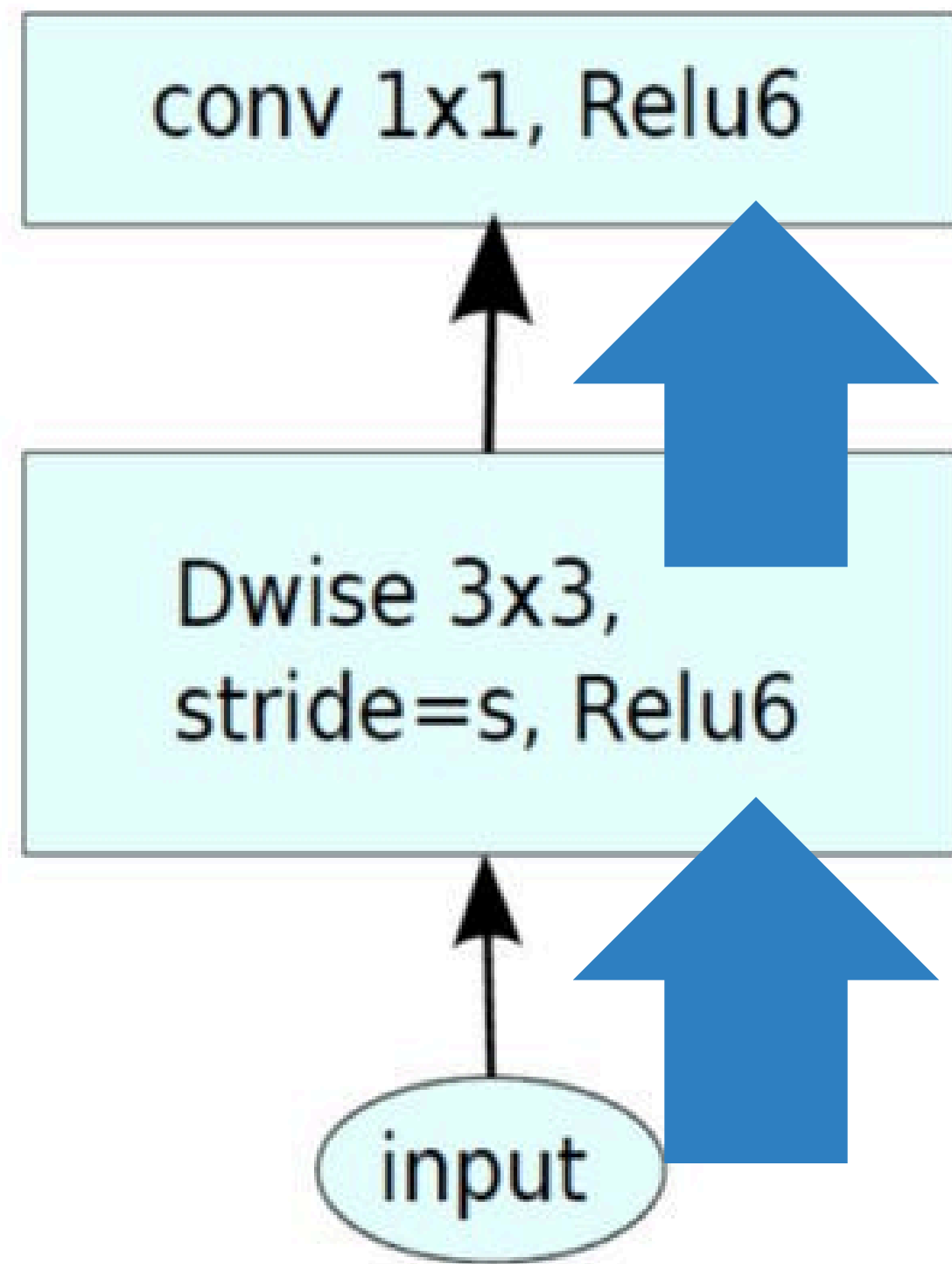
Stride=2 block

MobileNetV2

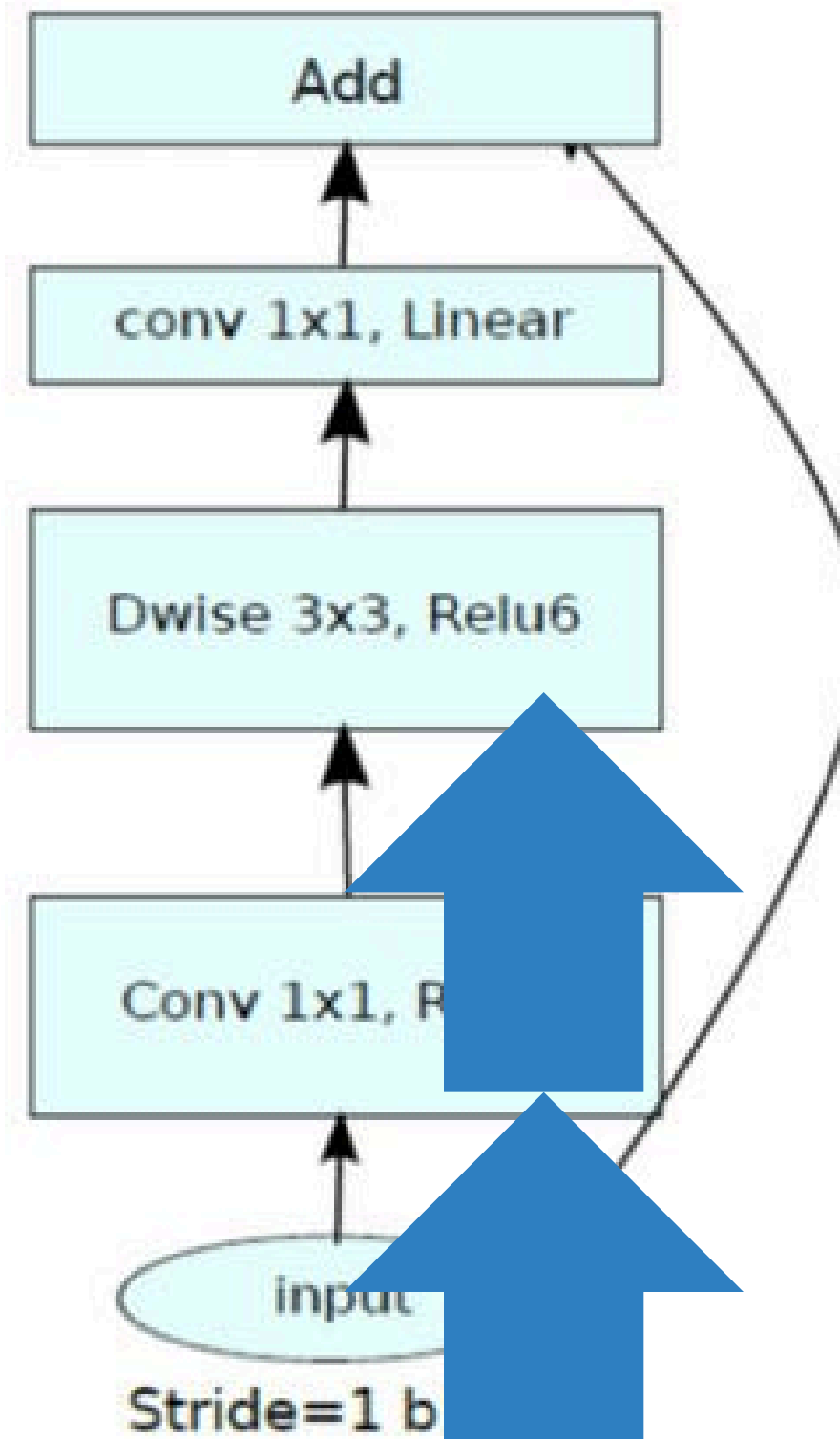
ReLU6

- Bu fonksiyon giriş tensöründeki her elemanı tek tek okuyup negatif olanları sıfıra, altıdan büyük olanları altıya kırparak ReLU6 aktivasyonunu uygular.
- Sonuç olarak girişle aynı boyutta yeni bir tensör oluşturur ve değerleri bu sınırlanmış aktivasyon fonksiyonundan geçirilmiş halde döndürür.

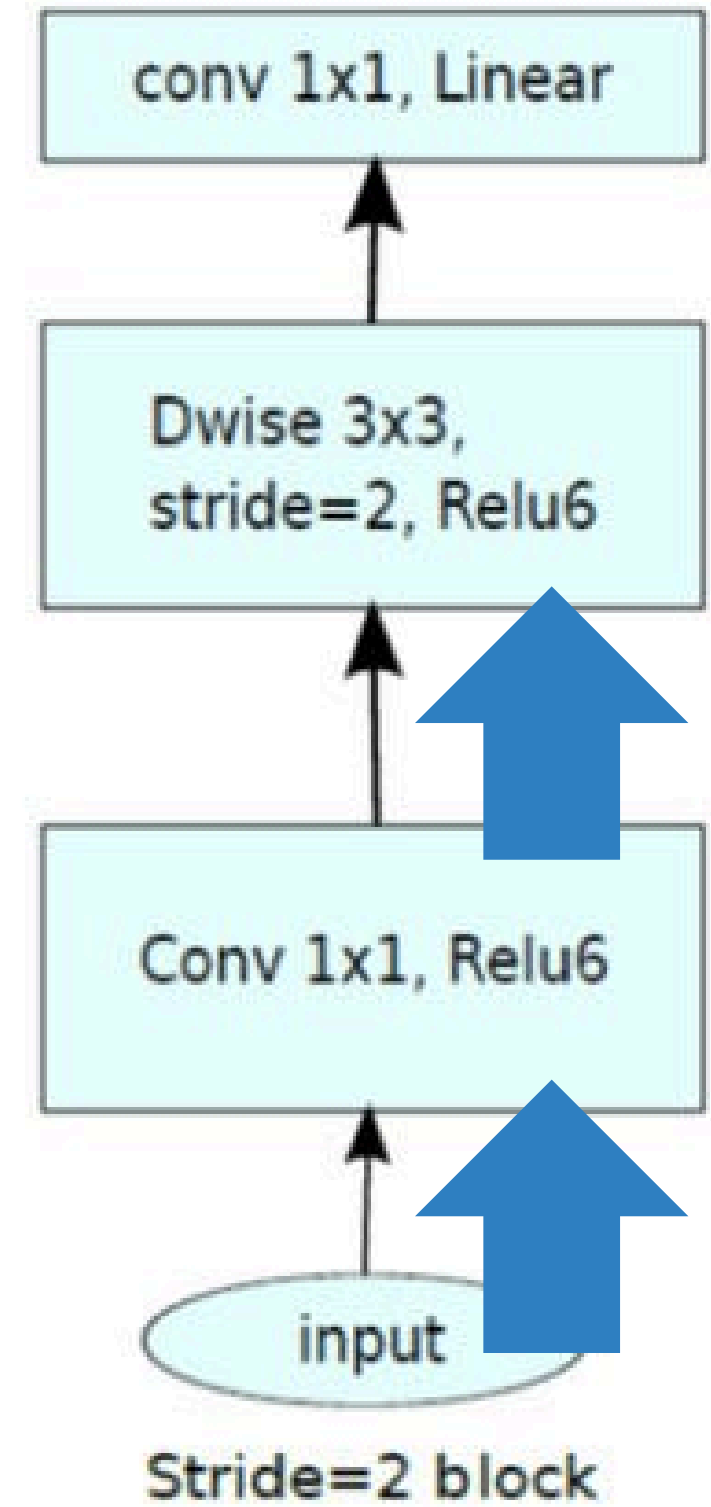
```
1  #include "ReLU6.h"
2  #include "Tensor.h"
3  #include <algorithm>
4
5  Tensor relu6_forward(const Tensor& x) {
6      Tensor y(x.c, x.h, x.w);
7
8      int size = x.c * x.h * x.w;
9      for (int i = 0; i < size; i++) {
10         float v = x.data[i];
11         if (v < 0.0f) v = 0.0f;
12         if (v > 6.0f) v = 6.0f;
13         y.data[i] = v;
14     }
15
16     return y;
17 }
18
```



MobileNetV1



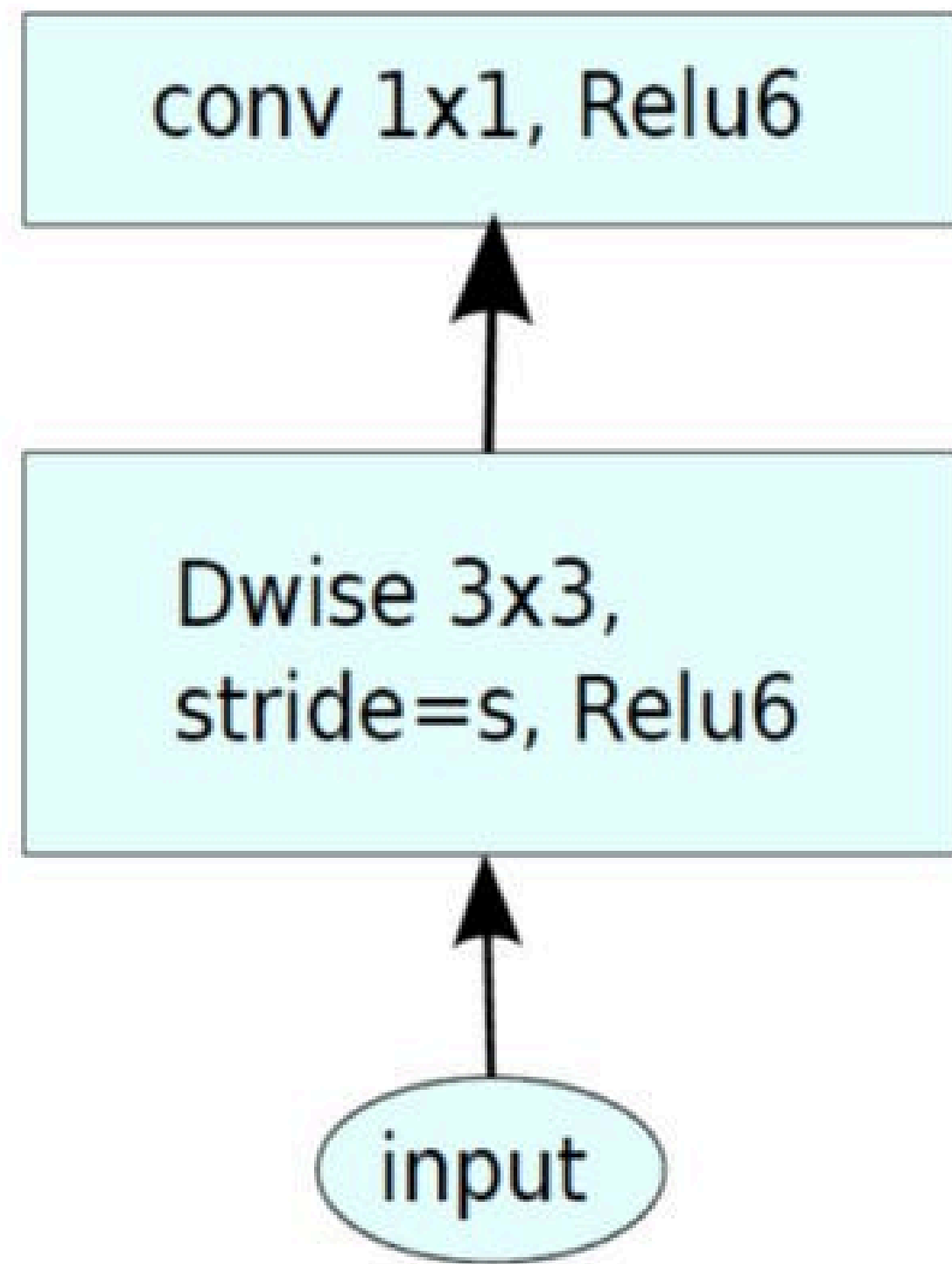
MobileNetV2



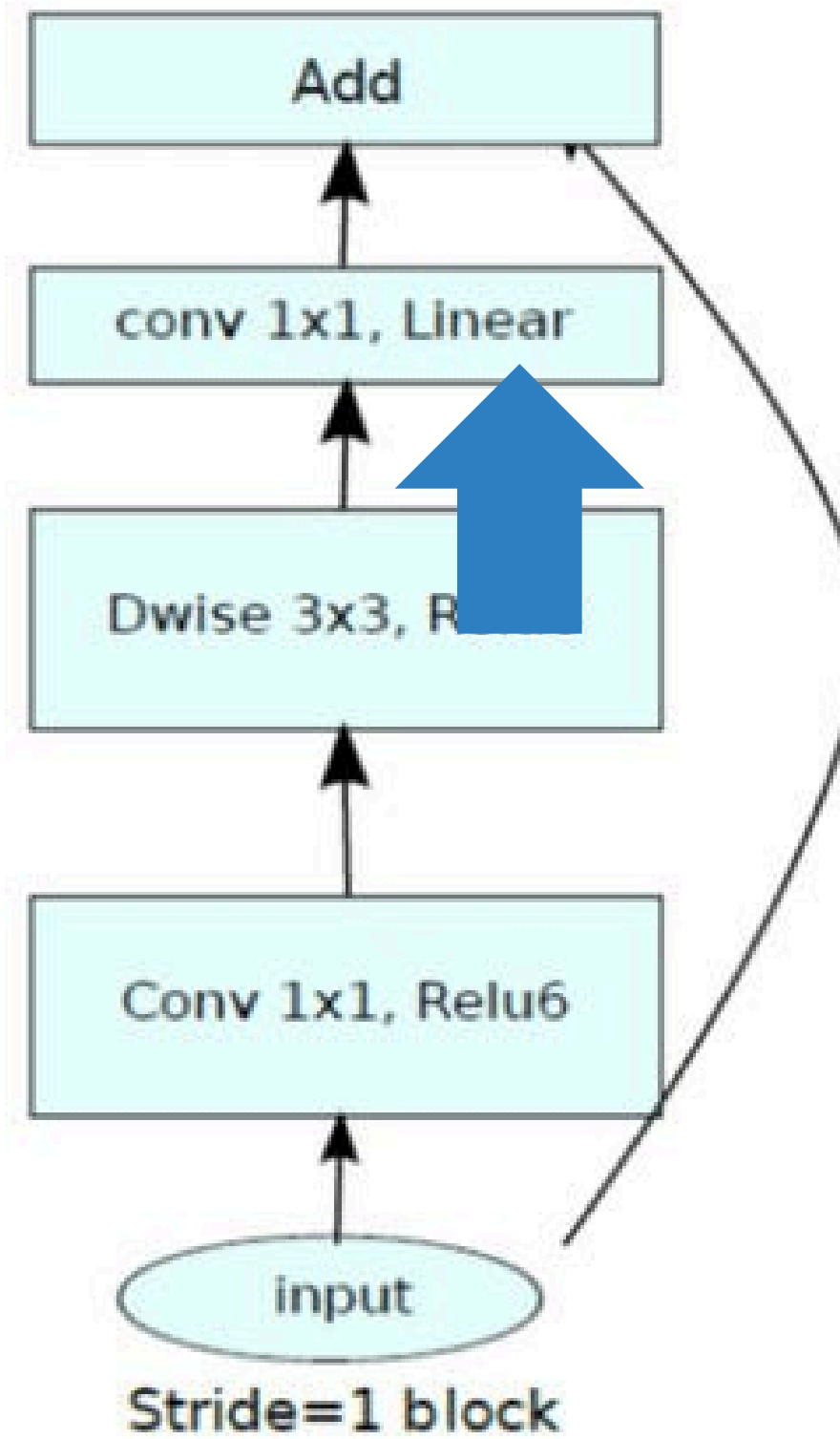
Linear

- Girdi tensörünü (in_features boyutunda) her çıkış nöronu için ağırlık matrisi ile çarpıp bias ekleyerek tam bağlı bir katmanın lineer dönüşümünü hesaplar.
- Sonuç olarak boyutu out_features olan yeni bir çıktı tensörü üretir, yani modelin sınıflandırma katmanındaki skorları oluşturur.

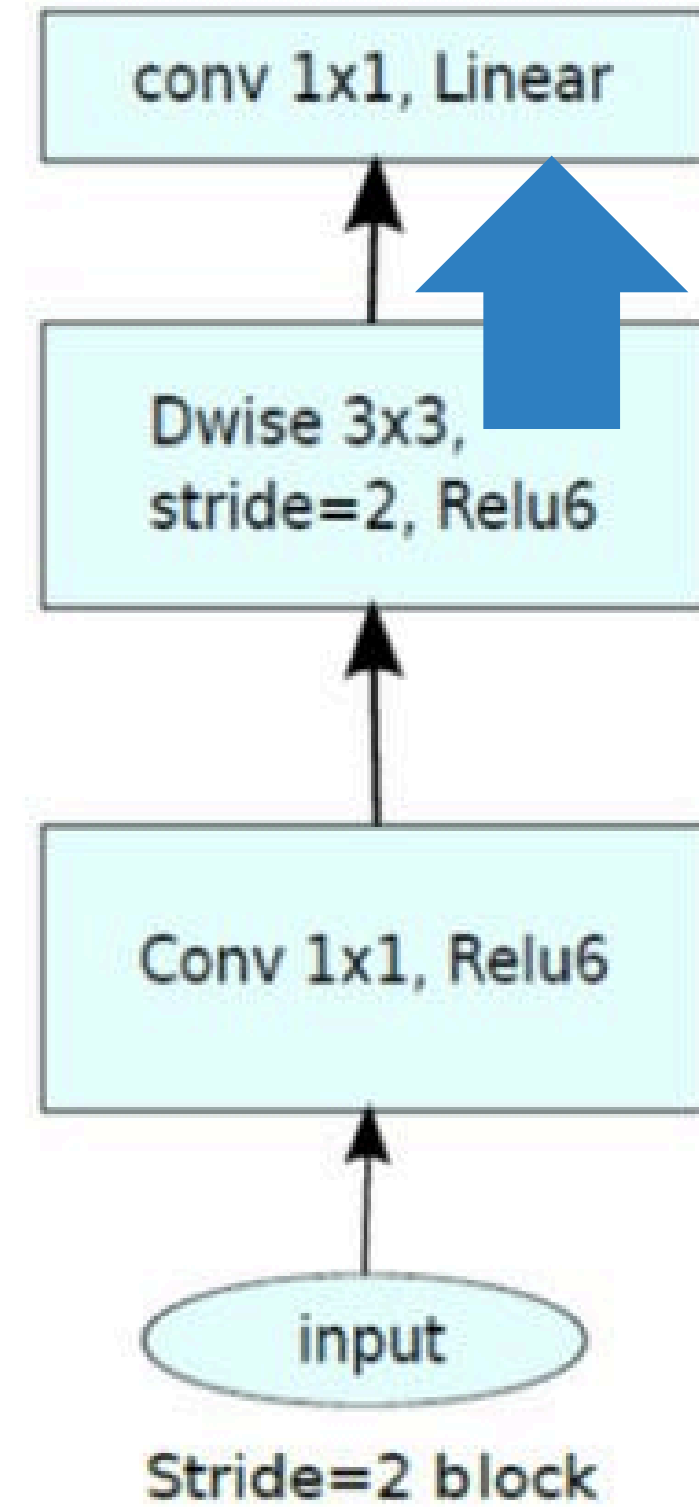
```
1  #include "Linear.h"
2  #include "Tensor.h"
3
4  Tensor linear_forward(const Tensor& x, const Linear& fc) {
5      int in_f = fc.getInFeatures();
6      int out_f = fc.getOutFeatures();
7
8      // x must be [in_f, 1, 1]
9      assert(x.c == in_f);
10
11     Tensor y(out_f, 1, 1);
12
13     const float* W = fc.getWeight(); // shape [out_f, in_f]
14     const float* B = fc.getBias();
15
16     for (int o = 0; o < out_f; o++) {
17         float sum = B[o];
18         for (int i = 0; i < in_f; i++) {
19             sum += W[o * in_f + i] * x.data[i];
20         }
21         y.data[o] = sum;
22     }
23
24     return y;
25 }
26
```



MobileNetV1



Stride=1 block



Stride=2 block

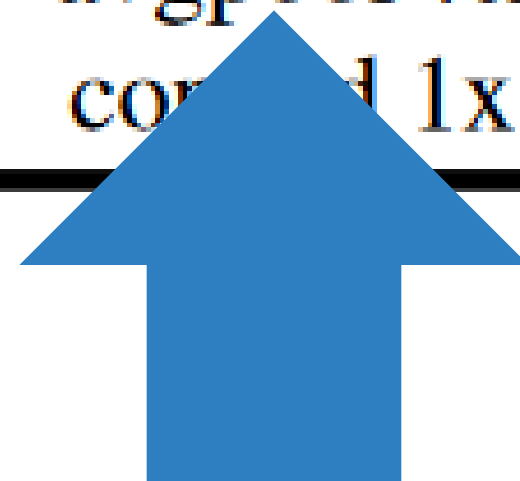
MobileNetV2

Global Average Pooling

- Her kanalın içindeki tüm piksellerin ortalamasını alıp tek bir değere indiriyor, yani $[C, H, W]$ şeklindeki bir feature mapi $[C, 1, 1]$ şekline sıkıştırıyor.
- Böylece CNN sonundaki sınıflandırıcıya sabit boyutlu, kompakt bir global temsil üretiyor, model parametre sayısını azaltıyor ve overfittingi düşürüyor.

```
mobilenet_v2 / C - GlobalAveragePool.cpp
1  #include "GlobalAveragePool.h"
2  #include "Tensor.h"
3
4  Tensor gap_forward(const Tensor& x) {
5      int C = x.c;
6      int H = x.h;
7      int W = x.w;
8
9      Tensor y(C, 1, 1); // output shape = [C, 1, 1]
10
11     for (int c = 0; c < C; c++) {
12         float sum = 0.0f;
13         for (int h = 0; h < H; h++) {
14             for (int w = 0; w < W; w++) {
15                 sum += x.data[c * H * W + h * W + w];
16             }
17         }
18         y.data[c] = sum / (H * W);
19     }
20
21     return y;
22 }
23
```

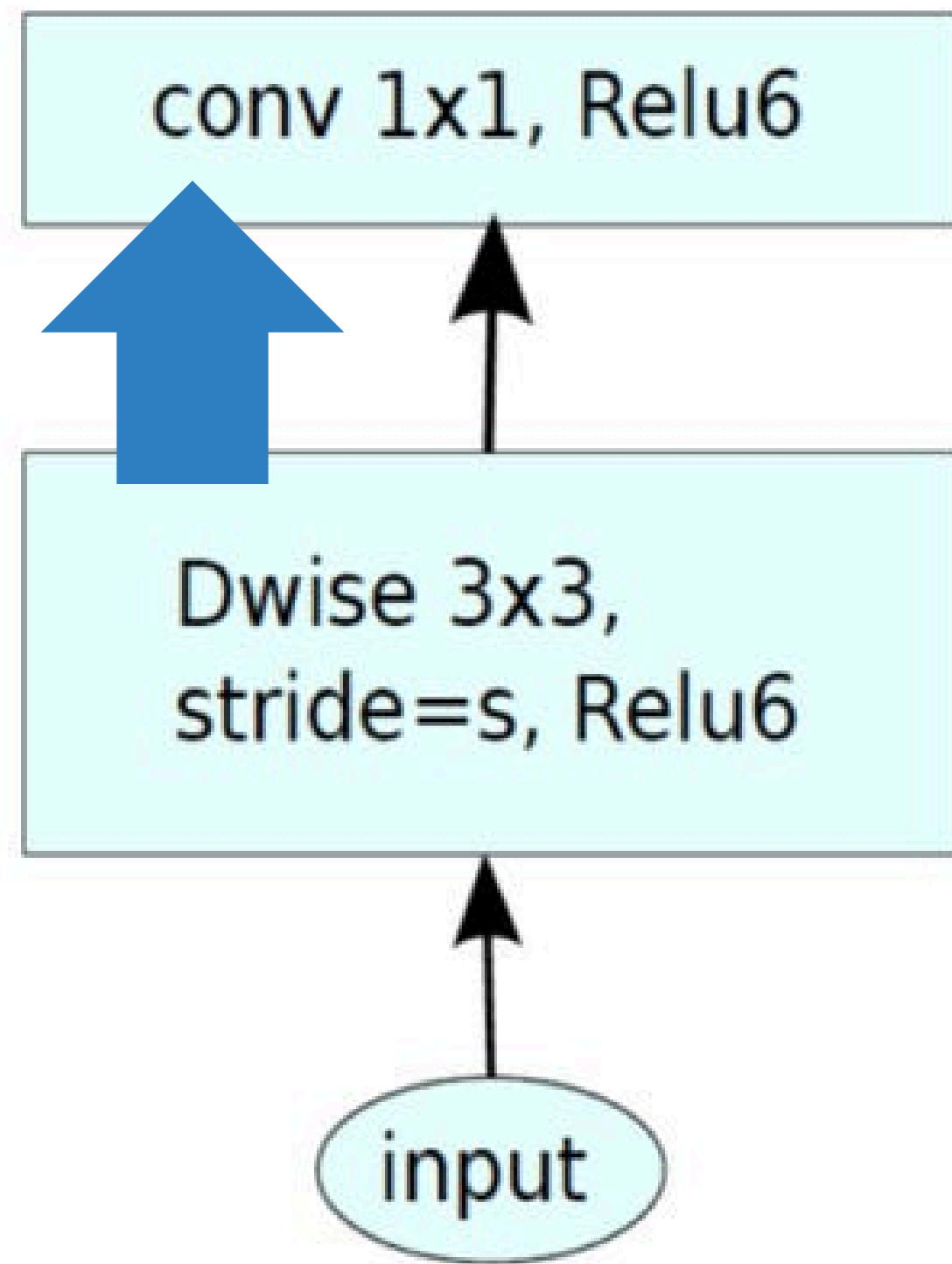
Input	Operator	t	c	n	s
$224^2 \times 3$	conv2d	-	32	1	2
$112^2 \times 32$	bottleneck	1	16	1	1
$112^2 \times 16$	bottleneck	6	24	2	2
$56^2 \times 24$	bottleneck	6	32	3	2
$28^2 \times 32$	bottleneck	6	64	4	2
$14^2 \times 64$	bottleneck	6	96	3	1
$14^2 \times 96$	bottleneck	6	160	3	2
$7^2 \times 160$	bottleneck	6	320	1	1
$7^2 \times 320$	conv2d 1x1	-	1280	1	1
$7^2 \times 1280$	avgpool 7x7	-	-	1	-
$1 \times 1 \times 1280$	conv2d 1x1	-	k	-	-



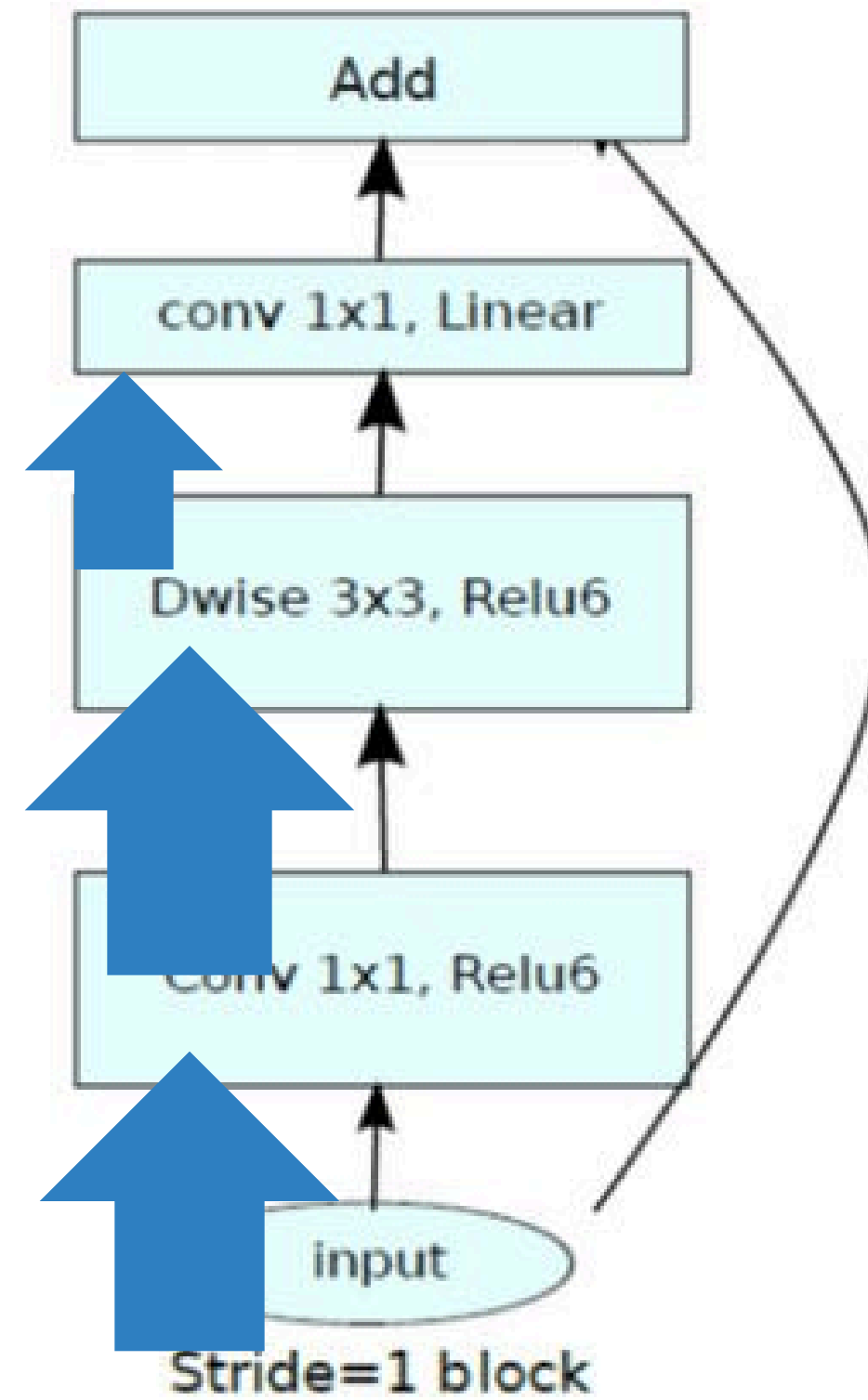
Convolutional 2D

- Kernel boyutu, stride, padding ve group sayısına göre her output pikseli için tüm ilgili input pikselleri ve ağırlıklar üzerinden çarp toplama yapıyor.

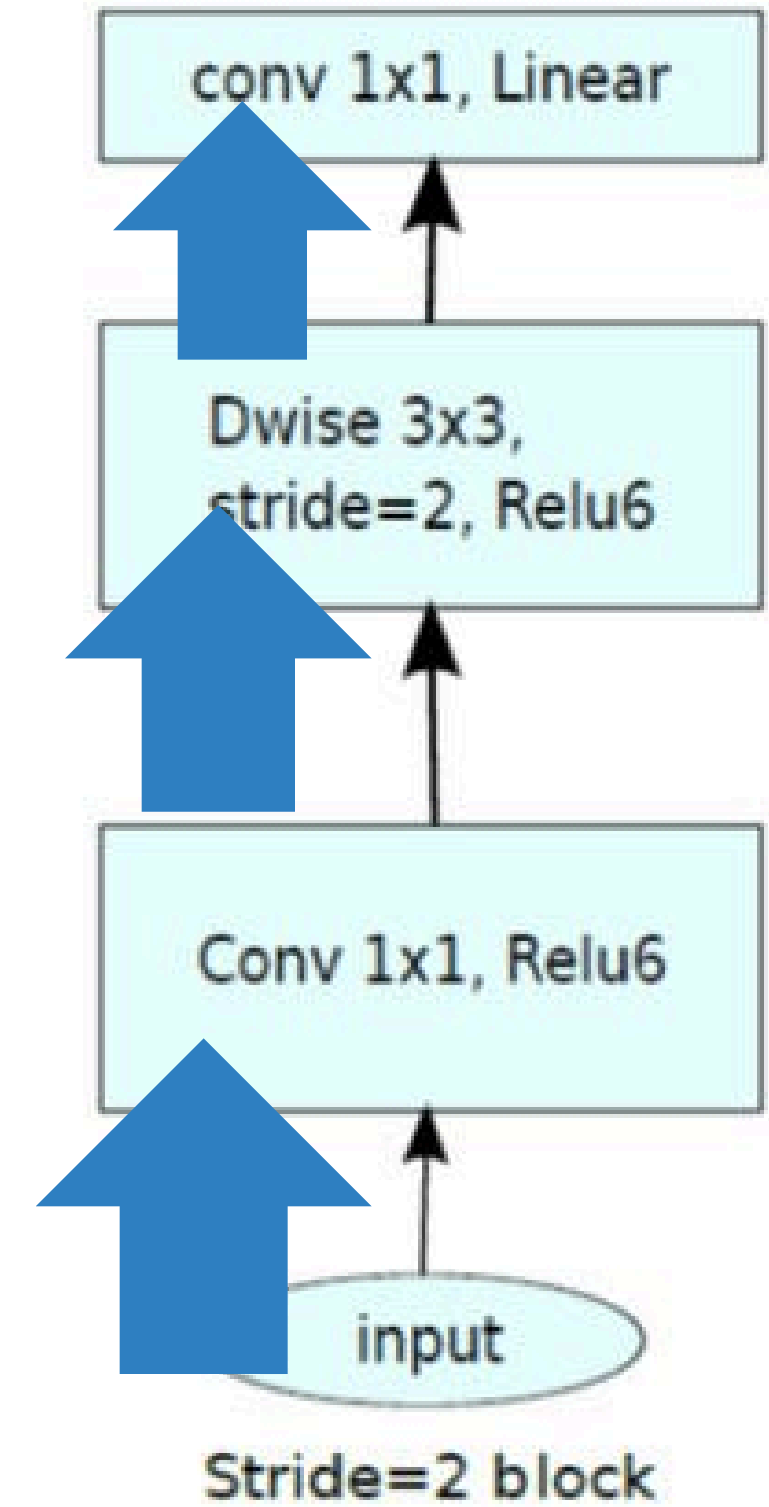
```
10 int Cin_per_group = layer.getInChannels(); // ctor içinde in_channels / group
11 int groups       = layer.getGroup();
12 int Cin_total    = Cin_per_group * groups; // gerçek input channel sayısı
13 int Cout         = layer.getOutChannels();
14 int K            = layer.getKernelSize();
15 int S            = layer.getStride();
16 int P            = layer.getPadding();
17
18 int Hin = x.h;
19 int Win = x.w;
20
21 int Hout = (Hin + 2 * P - K) / S + 1;
22 int Wout = (Win + 2 * P - K) / S + 1;
23
24 Tensor y(Cout, Hout, Wout);
25
26 const float* weight = layer.getWeight(); // shape: [Cout, Cin_per_group, K, K]
27 const float* bias    = layer.getBias();
28
29 // Her output channel için
30 for (int oc = 0; oc < Cout; ++oc) {
31     // Hangi gruptayız
32     int out_per_group = Cout / groups;
33     int g = oc / out_per_group; // group index
34     // Bu grubun input channel aralığı
35     int cin_start = g * Cin_per_group;
36     int cin_end   = cin_start + Cin_per_group;
37
38     for (int oh = 0; oh < Hout; ++oh) {
39         for (int ow = 0; ow < Wout; ++ow) {
40
41             float sum = bias[oc];
42
43             // Kernel içi dolaş
44             for (int ic = cin_start; ic < cin_end; ++ic) {
45                 int ic_local = ic - cin_start; // weight için local index
46
47                 for (int kh = 0; kh < K; ++kh) {
48                     for (int kw = 0; kw < K; ++kw) {
49                         int ih = oh * S + kh - P;
50                         int iw = ow * S + kw - P;
51
52                         if (ih < 0 || ih >= Hin || iw < 0 || iw >= Win) {
53                             continue; // padding bölgesi
54                         }
55
56                         int x_index = idx(ic, ih, iw, Cin_total, Hin, Win);
57                         int w_index = ((oc * Cin_per_group + ic_local) * K + kh) * K + kw;
58
59                         sum += x.data[x_index] * weight[w_index];
60                     }
61                 }
62             }
63         }
```

MobileNetV1



Stride=1 block



Stride=2 block

MobileNetV2

Cuda Operations

- Tüm MobileNetV2 katmanlarının GPU üzerinde çalışan CUDA kernel implementasyonlarını içerir, yani CPU versiyonundaki conv, relu6, residual add, global average pooling ve linear işlemlerinin tamamını GPUya taşır.
- Her katman için paralel kernel çalıştırarak hesaplamaları hızlandırır ve gerekli veri transferlerini (cudaMemcpy, cudaMalloc) yöneterek forward pass sırasında C++ modelinin GPU tarafında çalışmasını sağlar.

```
Tensor conv2d_forward_cuda(const Tensor& x, const Conv2d& layer) {  
    int Cin_per_group = layer.getInChannels();  
    int groups        = layer.getGroup();  
    int Cin_total     = Cin_per_group * groups;  
    int Cout          = layer.getOutChannels();  
    int K             = layer.getKernelSize();  
    int S             = layer.getStride();  
    int P             = layer.getPadding();  
  
    int Hin = x.h;  
    int Win = x.w;  
  
    int Hout = (Hin + 2 * P - K) / S + 1;  
    int Wout = (Win + 2 * P - K) / S + 1;  
  
    Tensor y(Cout, Hout, Wout);  
  
    const float* h_w = layer.getWeight();  
    const float* h_b = layer.getBias();  
  
    size_t in_size  = Cin_total * Hin * Win * sizeof(float);  
    size_t out_size = Cout * Hout * Wout * sizeof(float);  
    size_t w_size   = layer.getSize() * sizeof(float);  
    size_t b_size   = Cout * sizeof(float);  
  
    float *d_x, *d_y, *d_w, *d_b;  
    cudaMalloc(&d_x, in_size);  
    cudaMalloc(&d_y, out_size);  
    cudaMalloc(&d_w, w_size);  
    cudaMalloc(&d_b, b_size);  
  
    cudaMemcpy(d_x, x.data, in_size, cudaMemcpyHostToDevice);  
    cudaMemcpy(d_w, h_w, w_size, cudaMemcpyHostToDevice);  
    cudaMemcpy(d_b, h_b, b_size, cudaMemcpyHostToDevice);  
  
    int total = Cout * Hout * Wout;  
    int block = 256;  
    int grid  = (total + block - 1) / block;  
  
    conv2d_kernel<<<grid, block>>>(  
        d_x, d_w, d_b, d_y,  
        Cin_total, Cin_per_group, Cout,  
        Hin, Win, Hout, Wout,  
        K, S, P, groups  
    );  
    cudaDeviceSynchronize();  
  
    cudaMemcpy(y.data, d_y, out_size, cudaMemcpyDeviceToHost);  
  
    cudaFree(d_x);  
    cudaFree(d_y);  
}
```


- Tensorlar hosttan device'a, sonuçlar device'tan hosta taşınıyor ve bütün computation GPU üzerinden ilerliyor.

```
Tensor MobileNetV2::forward(const Tensor& input) {  
  
    std::unordered_map<Layer*, Tensor> tensor_map;  
  
    Tensor x = input;  
    Layer* layer = entry;  
  
    while (layer != nullptr) {  
        int t = layer->getType();  
  
        if (t == 0) {  
            auto* conv = (Conv2d*)layer;  
            x = conv2d_forward_cuda(x, *conv);  
        } else if (t == 3) {  
            x = relu6_forward_cuda(x);  
        } else if (t == 4) {  
            auto* add = (ResidualAdd*)layer;  
            Layer* skip_layer = add->getResidual();  
            Tensor skip_tensor = tensor_map[skip_layer];  
            x = residual_add_forward_cuda(x, skip_tensor);  
        } else if (t == 1) {  
            x = gap_forward_cuda(x);  
        } else if (t == 2) {  
            auto* fc = (Linear*)layer;  
            x = linear_forward_cuda(x, *fc);  
        }  
  
        tensor_map[layer] = x;  
        layer = layer->getNext();  
    }  
  
    return x;  
}
```


Benchmarking Pipeline:

- Her resmi yükleyip 224×224 şekline resize ettim
- ImageNet normalization uyguladım.

```
1  #include <iostream>
2  #include <filesystem>
3  #include <vector>
4  #include <chrono>
5  #include <algorithm>
6  #include <fstream>
7
8  #include "mobileNetCuda.h"
9  #include "Tensor.h"
10
11 #define STB_IMAGE_IMPLEMENTATION
12 #include "stb_image.h"
13
14 namespace fs = std::filesystem;
15
16 // ImageNet normalization
17 inline void normalize(float& r, float& g, float& b) {
18     r = (r - 0.485f) / 0.229f;
19     g = (g - 0.456f) / 0.224f;
20     b = (b - 0.406f) / 0.225f;
21 }
22
23 // Simple nearest neighbor resize
24 void resize224(const unsigned char* input, int w, int h, float* out) {
25     for (int y = 0; y < 224; y++) {
26         for (int x = 0; x < 224; x++) {
27
28             int src_x = x * w / 224;
29             int src_y = y * h / 224;
30             int idx_input = (src_y * w + src_x) * 3;
31
32             float r = input[idx_input] / 255.0f;
33             float g = input[idx_input + 1] / 255.0f;
34             float b = input[idx_input + 2] / 255.0f;
35
36             normalize(r, g, b);
37
38             out[idx(0, y, x, 3, 224, 224)] = r;
39             out[idx(1, y, x, 3, 224, 224)] = g;
40             out[idx(2, y, x, 3, 224, 224)] = b;
41         }
42     }
43 }
```

Benchmarking Pipeline:

- CUDA MobileNetV2 modelimde forward pass yaptım.
- Her forward için latency ölçtüm
- Top 1 ve Top 5 accuracy hesapladım.
- Genel throughput ve ortalama gecikme sürelerini raporladım.

```
// -----  
// Forward latency measurement  
// -----  
auto f0 = std::chrono::high_resolution_clock::now();  
Tensor out = model.forward(input);  
auto f1 = std::chrono::high_resolution_clock::now();  
forward_total_time += std::chrono::duration<double>(f1 - f0).count();  
  
// Top5 accuracy  
std::vector<std::pair<float, int>> scores;  
scores.reserve(1000);  
for (int i = 0; i < 1000; i++)  
    scores.emplace_back(out.data[i], i);  
  
std::partial_sort(scores.begin(), scores.begin() + 5, scores.end(),  
    [](auto& a, auto& b) { return a.first > b.first; });  
  
int pred1 = scores[0].second;  
  
bool top5_hit = false;  
for (int k = 0; k < 5; k++)  
    if (scores[k].second == class_label) top5_hit = true;  
  
if (pred1 == class_label) correct_top1++;  
if (top5_hit) correct_top5++;  
  
total_images++;  
  
if (total_images % 100 == 0)  
    std::cout << "Processed: " << total_images << "\n";  
}  
}  
  
auto t1 = std::chrono::high_resolution_clock::now();  
double total_elapsed = std::chrono::duration<double>(t1 - t0).count();  
  
double throughput = total_images / total_elapsed;  
double avg_latency = (forward_total_time / total_images) * 1000.0; // ms  
  
std::cout << "Images: " << total_images << "\n";  
std::cout << "Top 1 accuracy: " << (100.0 * correct_top1 / total_images) << "\n";  
std::cout << "Top 5 accuracy: " << (100.0 * correct_top5 / total_images) << "\n";  
std::cout << "Total time: " << total_elapsed << " sec\n";  
std::cout << "Throughput: " << throughput << " images per sec\n";  
  
std::cout << "Average latency per image: " << avg_latency << " ms\n";
```

```
Images: 50000  
Top 1 accuracy: 0.084  
Top 5 accuracy: 0.59  
Total time: 287.024 sec  
Throughput: 174.201 images per sec  
Average latency per image: 3.51641 ms
```

Model tüm forward pipeline ile başarıyla çalıştı ve 50 bin görüntü üzerinde benchmark edildi. GPU üzerinde ortalama 3.5 ms latency ve 174 img/sn throughput elde ettim.

Pretrained ağırlıkları doğru okuduğumu top 5 accuracy değerinin %59 olması gösteriyor ama top 1 accuracy çok düşük ona bakacağım.

Gelecek Hafta Hedeflerim

- cuDNN ve cuBLAS ile entegrasyon: Mevcut forward kodunu, cuDNN Conv ve cuBLAS Gemm gibi hazır GPU kütüphaneli primitiflerle hızlandıracam.
- Özel CUDA kernel yapısına geçiş: Conv, Gemm ve Pool operasyonları için kendi optimize edilmiş CUDA kernel versiyonlarımı yazıp test edeceğim.
- Hız karşılaştırması ve optimizasyon: Mevcut CUDA ve bu kütüphanelerle optimize edilmiş sürümü ölçüp karşılaştıracam ve gerekiyorsa kernel bazlı iyileştirmeler yapacağım.